KUBERNETES ATTACK PATH GRAPH USING NEO4J AND GNN

A PROJECT REPORT

Submitted by

JASON FREDDY F.P. 231901014

SABTHA GIRI A. 231901044

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 02

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled " **KUBERNETES ATTACK GRAPH USING NEO4J AND GRAPH NEURAL NETWORK"** is the bonafide work of **"JASON FREDDY F P (2116231901014) , SABTHA GIRI A (2116231901044) "** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

7/10/26

SIGNATURE

Dr. P. Tamilselvi, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

7/10/26

SIGNATURE

Dr. P. Tamilselvi, M.E., Ph.D.,

SUPERVISOR

PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 7/10/2026

Internal Examiner

7/10/26

External Examiner

ABSTRACT

This project presents a graph-based security analysis tool for Kubernetes clusters. Kubernetes environments contain interconnected resources such as Pods, Deployments, Services, ServiceAccounts, Roles and ClusterRoles. A security weakness in one resource can therefore become more serious when combined with network exposure, excessive permissions or host-level privileges. Traditional configuration scanners commonly report individual findings, but they do not always show how multiple weaknesses can form a practical attack chain. The proposed K8s Attack Graph system addresses this problem by discovering relevant Kubernetes resources, representing their relationships as a graph, identifying privilege-risk paths, computing transparent per-Pod risk scores and optionally refining those scores using a Graph Convolutional Network (GCN).

The implemented system uses the Kubernetes Python client for cluster discovery, Redis for caching the latest discovery result, Neo4j for persistent graph representation, NetworkX for graph traversal and path analysis, FastAPI for the backend API, PyTorch for the GCN module, and React with Cytoscape.js for visualization. The graph represents relationships including RUNS_AS, BOUND_TO, EXPOSES and MANAGES. The attack-path engine identifies paths from Pods to dangerous permissions such as cluster-admin, wildcard permissions, privilege-escalation verbs, pod-exec capability and secrets access. The rule-based risk engine produces a 0–100 score using RBAC escalation potential, network exposure, container/host privilege, identity/image hygiene and graph centrality.

A small two-layer GCN is implemented as a proof-of-concept re-ranking layer. It uses graph-derived node features and combines manually supplied ground-truth labels with rule-based scores as pseudo-labels. The project also

includes a pretrained initialization based on RBAC Atlas data, while explicitly treating the GNN as an enhancement to the rule-based baseline rather than as an independently validated production risk predictor. The end-to-end implementation was verified using a local Minikube demonstration cluster containing intentionally misconfigured workloads and a safe control case. The resulting dashboard provides an overview, interactive graph, risk table, attack-path view and GNN comparison.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** for her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

JASON FREDDY F.P. - 231901014

SABTHA GIRI A. - 231901044

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**3**
	LIST OF TABLES	**7**
	LIST OF FIGURES	**8**
1	**INTRODUCTION**	**9**
	1.1 BACKGROUND	9
	1.2 PROBLEM STATEMENT	9
	1.3 OBJECTIVES	9
	1.4 SCOPE	10
	1.5 SECURITY REQUIREMENTS	10
	1.6 SDG ALIGNMENT	11
2	**LITERATURE REVIEW**	**12**
	2.1 EXISTING SYSTEM	12
	2.2 RESEARCH GAPS	21
	2.3 PROPOSED SYSTEM	21
	2.3.1 OVERVIEW	21
	2.3.2 OBJECTIVES	21
	2.3.3 ADVANTAGES	21
3	**SYSTEM DESIGN**	**22**
	3.1 SYSTEM ARCHITECTURE	22
	3.2 MODULE DESIGN	23
	3.2.1 CLUSTER DISCOVERY MODULE	23
	3.2.2 ATTACK GRAPH GENERATOR	23
	3.2.3 ATTACK PATH ENGINE	24
	3.2.4 RISK ENGINE	24
	3.2.5 GNN RE-RANKING MODULE	24

CHAPTER NO.	TITLE	PAGE NO.
	3.2.6 FRONTEND AND API MODULE	24
	3.3 DATA FLOW	24
	3.4 THREAT MODEL AND ATTACK SURFACE	27
	3.5 SECURITY CONTROLS	27
4	**SYSTEM DESCRIPTION**	**28**
	4.1 DATA COLLECTION	28
	4.2 DATA PREPROCESSING	28
	4.3 METHODOLOGY	29
	4.3.1 GRAPH CONSTRUCTION	29
	4.3.2 ATTACK PATH SEARCH	29
	4.3.3 RULE-BASED RISK SCORING	29
	4.3.4 GCN RE-RANKING	30
	4.4 RESULTS AND DISCUSSION	30
	4.4.1 DEMO CLUSTER SCENARIOS	34
	4.4.2 OBSERVED END-TO-END OUTPUTS	34
5	**CONCLUSION**	**35**
	REFERENCES	**36**
	APPENDIX	**38**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SDG ALIGNMENT MAPPING	11
2.1	EXISTING SYSTEM WITH SECURITY APPROACHES	13
3.1	MODULES AND TECHNOLOGIES	23
3.2	ACTORS AND COMPONENT INTERACTIONS	25
3.3	MISCONFIGURATION AND GRAPH SIGNALS	27
4.1	COLLECTED KUBERNETES RESOURCES	28
4.2	RISK-SCORING FACTORS AND WEIGHTS	29
4.3	DEMO CLUSTER SECURITY SCENARIOS	34
4.4	OBSERVED END-TO-END OUTPUTS	34

LIST OF FIGURES

FIGURE	TITLE	PAGE NO.
3.1	SYSTEM ARCHITECTURE	22
3.2	ACTIVITY FLOW OF THE ANALYSIS PIPELINE	26
4.1	DASHBOARD OVERVIEW	30
4.2	GENERATED KUBERNETES ATTACK GRAPH	31
4.3	POD RISK ANALYSIS	31
4.4	ATTACK PATH VIEW	32
4.5	GNN RE-RANKING VIEW	33

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Kubernetes has become a common platform for deploying containerized applications, but its flexibility also creates a large security configuration space. A cluster may contain many workloads, ServiceAccounts, RBAC bindings, Services and administrative permissions. These resources are not independent. A Pod exposed through a Service may run with a ServiceAccount that is bound to a powerful ClusterRole, while a privileged container or host namespace setting may provide an additional route toward node-level compromise.

The project therefore treats Kubernetes security as a relationship-analysis problem rather than only a list-of-findings problem. The central idea is to construct an attack graph in which Kubernetes resources become nodes and security-relevant relationships become directed edges. Once the cluster is represented this way, graph algorithms can identify chains that are difficult to understand from isolated configuration checks.

The completed implementation follows a staged pipeline. Cluster Discovery retrieves the current inventory using the Kubernetes Python client. Redis stores the latest discovery result so subsequent steps can reuse it. The Attack Graph Generator converts the inventory into a Neo4j graph. The Attack Path Engine reads the graph into NetworkX and searches for paths to dangerous permissions. The Risk Engine assigns transparent 0–100 scores to Pods. Finally, the GNN module provides an optional learned re-ranking layer, while the React/Cytoscape dashboard makes the analysis accessible for demonstration and review.

1.2 PROBLEM STATEMENT

Existing Kubernetes security analysis approaches can identify misconfigurations, excessive permissions and exposed services, but an analyst may still need to manually connect individual findings to understand a realistic escalation chain. The problem addressed by this project is therefore: how can Kubernetes configuration, RBAC relationships, network exposure and host/container security signals be combined into an interpretable graph so that privilege-risk paths and per-Pod risk can be identified automatically?

1.3 OBJECTIVES

- Discover Pods, Deployments, Services, ServiceAccounts, Roles, ClusterRoles and their bindings from a Kubernetes cluster.
- Represent the discovered resources and security relationships as a graph in Neo4j.
- Traverse the graph using NetworkX to identify shortest paths and notable privilege-risk paths.
- Detect dangerous RBAC patterns including cluster-admin, wildcard permissions, privilege-escalation verbs, pod exec capability and secrets access.
- Compute a transparent 0–100 Pod risk score using exposure, RBAC escalation, container/host privilege, hygiene and graph centrality.
- Provide a dashboard for viewing the graph, risk scores, attack paths and GNN comparisons.
- Implement a small GCN as a proof-of-concept re-ranking layer on top of the rule-based baseline.

1.4 SCOPE

The project focuses on configuration and privilege-risk analysis within a Kubernetes cluster. It covers Kubernetes resource discovery, RBAC relationships, service exposure, selected container/host security settings, graph construction, graph traversal, risk scoring and GNN-based re-ranking. It is designed for local development and demonstration using Minikube, although the discovery module also supports standard Kubernetes configuration and in-cluster configuration loading.

The project does not attempt to perform exploitation, penetration testing or automated remediation. It also does not claim to be a complete vulnerability scanner. In particular, the current implementation does not wire a CVE or image-vulnerability scanner into the risk formula; the report therefore treats container/host privilege as a configuration-risk factor rather than presenting it as a CVSS measurement.

1.5 SECURITY REQUIREMENTS

- Read-only or least-privileged access should be used for discovery wherever the deployment environment permits it.
- Intentionally vulnerable demonstration manifests must only be deployed in an isolated, disposable cluster such as Minikube.

- Graph data may contain security-sensitive resource names and permission information and should be protected in a real deployment.
- Neo4j and Redis credentials must be changed before any non-local deployment; the supplied project uses development credentials for demonstration.
- API CORS configuration is permissive in the current development build and should be tightened for production.
- Risk scores are decision-support outputs, not proof of successful compromise; findings should be validated by an administrator or security analyst.
- The GNN is explicitly treated as a proof-of-concept refinement layer because its training data in the demonstration environment is limited.

1.6 SDG ALIGNMENT

TABLE 1.1 : SDG ALIGNMENT

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 9 – Industry, Innovation and Infrastructure	9.5 / technology and innovation	The project applies graph databases, cloud-native security concepts and machine learning to improve analysis of containerized infrastructure.
2	SDG 16 – Peace, Justice and Strong Institutions	16.6 / effective, accountable institutions	The system improves visibility into access relationships and security configuration, supporting accountable management of digital infrastructure.

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Kubernetes security can be approached through multiple complementary mechanisms. Kubernetes RBAC controls which identities can perform actions on resources. Configuration benchmarks and security scanners can identify insecure settings. Container and image scanners focus on vulnerabilities in software artifacts. Runtime and network security tools observe activity and traffic. These approaches are valuable, but their outputs are often organized around individual resources or findings.

RBAC analysis is especially important for this project because permissions can form transitive relationships. A Pod does not normally hold permissions directly; it operates under a ServiceAccount, which can be associated with a Role or ClusterRole through bindings. A graph representation naturally expresses this chain as Pod → ServiceAccount → Role/ClusterRole.

Graph databases are suited to relationship-heavy data because nodes and relationships are first-class entities. In this project, Neo4j stores the discovered cluster structure, while NetworkX provides convenient graph traversal in Python. The combination supports both persistent visualization and algorithmic path analysis.

Graph neural networks provide another way to incorporate local graph structure into node-level predictions. The implemented model follows the standard graph-convolution idea in which normalized adjacency is used to propagate feature information between connected nodes. The project uses this technique only as a re-ranking extension, keeping the rule-based engine as the transparent baseline.

TABLE 2.1 :EXISTING SYSTEM WITH SECURITY APPROACHES

#	Paper detail	Tools & techniques	Applications	Dataset	Advantages / Disadvantages	Inference for current project
1	XI Commandments of Kubernetes Security (Shamim, Bhuiyan, Rahman), IEEE SecDev 2020, pp. 58–64, Tennessee Tech. arXiv 2006.15275. A systematization of knowledge.	Qualitative analysis of Internet artifacts to synthesize security practices. No tool built.	Benchmark checklist for hardening Kubernetes. Groundwork for later misconfiguration studies.	A curated set of 104 Internet artifacts, mapped to practices.	+ Practitioner-sourced 11-practice taxonomy with a public artifact-to-practice mapping. Counts show which practices are cited most (authentication/authorization 82, K8s-specific policies 81, vulnerability scanning 63). – Grey literature, not tested on real deployments. Counts show how often a practice is cited, not how much it matters (inference).	Use the 11 practices as the best-practice baseline your rules are checked against. Rahman's TOSEM paper derives its misconfiguration categories from it.
2	Security Misconfigurations in Open Source Kubernetes Manifests (Rahman, Shamim, Bose, Pandita), ACM TOSEM 32(4), 2023.	Open coding to build the taxonomy. SLI-KUBE: Python static analyser using rule/pattern matching plus def-use chain analysis. Logistic regression, Mann-Whitney U, Krippendorff's α. Practitioner bug reports and 9 interviews.	Detecting 11 misconfiguration categories in Kind and Helm manifests (hostNetwork, privileged securityContext, hard-coded secret, insecure HTTP, etc.). Suited to CI integration.	2,039 manifests from 92 OSS repos (71 GitHub, 21 GitLab). Taxonomy built from 1,796 manifests (Bose et al.). Oracle set of 240 manifests.	+ First systematic taxonomy and large mining study. Average precision 0.9 and recall 1.0. Only tool covering all 11 categories versus Checkov, KubeLinter, Datree and Snyk. Found 1,051 misconfigurations. – Pattern-matching false positives (hard-coded secrets precision 0.82). No severity ranking. No service-mesh awareness. Static only. OSS-only data. Bug-report response just 4.1%.	High relevance. Source for your hygiene checks. Interviewees asked for severity ranking and live-cluster analysis, which your tool provides. Larger manifests had more misconfigurations, so prioritise them.

#	Paper detail	Tools & techniques	Applications	Dataset	Advantages / Disadvantages	Inference for current project
3	A Survey of Automatic Generation of Attack Trees and Attack Graphs (Konsta, Spiga, Lluch Lafuente, Dragoni), arXiv 2302.14479 (v3 Sept 2023), DTU.	Snowballing literature review, starting from a 5-paper seed and ending with 20 papers. Classifies techniques as Model-driven, Analysis-driven or Vulnerability-driven. Surveyed methods include model checking (NuSMV, JKind), PDDL planning, Datalog/MulVAL, graph reachability, templates and pattern matching, ML, grammar parsing.	Automatically generating attack trees and graphs for networks, SCADA, socio-technical systems, buildings and protocols.	No dataset. It is a literature survey.	+ First survey focused only on automated generation. Compares approaches on proofs, open source, reproducibility and scalability. – Finds state-space explosion, static templates, prerequisite burden, no reproducibility badges, little scalability evidence. Nothing Kubernetes-specific.	Use the 3-category framework to position your method. The limitations list gives your evaluation checklist (scalability, reproducibility, meaningful output).
4	Stay at the Helm (Blaise & Rebecchi), IEEE CLOUD, July 2022 (IEEE 9860863). Proposes a method to extract and evaluate the security model of Helm Charts.	Builds a topological graph of the chart whose nodes and edges carry security features, with risk scores tied to MITRE ATT&CK tactics, then pulls out the riskiest attack paths.	Pre-deployment risk assessment of Helm charts; attack-path reconstruction.	A dataset built from several public Helm chart repositories (size not confirmed).	+ Combines a graph model, ATT&CK-aligned scoring and attack paths. Finds that most charts have vulnerabilities exploitable through complex attack paths. – Full text not read. Likely static only, so it cannot see runtime behaviour (inference, not stated by the authors).	Closest conceptual baseline for "chart → graph → attack paths".
5	KubeHound (Dell'Immagine, Soldani, Brogi), Future Internet 15(7):228, June 2023, Univ. of Pisa.	Static plus dynamic analysis. Plugin architecture (Hound, Frontend, Scheduler) in Python, open source. Wraps existing tools: Kubesec.io, Checkov, Kube-hunter, Nmap, SonarQube, Ksniff/tshark, detect-secrets, OpenAPI-fuzzer. Targets the 10 microservice "security smells" of Ponce et al.	Auditing microservice deployments for security smells. Six built-in plugins (Kubesec, detect-secrets, Ksniff, OpenAPI analysis, etc.).	A mock 7-microservice app with injected smells, plus two third-party apps: Sock Shop (14 services) and Online Boutique (11 services, with and without Istio).	+ Extensible and "best effort" (runs only what inputs allow). Reuses mature tools. Found a real hardcoded DB password in Sock Shop. – Only 6 plugins implemented. Heuristic false positives (secrets; Kubesec only reads container-level securityContext). No confidence score, and no precision or recall figures. Only 3 apps tested.	Medium. Sock Shop and Online Boutique are ready-made test apps beyond your 4 demo manifests. It shows Kubesec missed runAsNonRoot set at pod level rather than container level. Handle both levels in your new hygiene checks.

#	Paper detail	Tools & techniques	Applications	Dataset	Advantages / Disadvantages	Inference for current project
6	Inside Job: Defending Kubernetes Clusters Against Network Misconfigurations (Bufalino, Martin-Navarro, Di Francesco, Aura), arXiv 2506.21134 (June 2025). Also listed in ACM (DOI 10.1145/3749220); Aalto University among the affiliations.	Literature review plus thematic analysis to define misconfigurations M1–M7. Hybrid static + runtime analysis: a custom YAML parser, plus Helm installs on Minikube with observed open ports, checked against machine-readable rules (Minikube 1.23, K8s 1.25). Compared with 11 existing tools. Threat model based on Microsoft's K8s matrix and MITRE lateral movement.	Detecting network misconfigurations that enable lateral movement (undeclared or dynamic ports, label collisions, missing network policies, hostNetwork). Two proof-of-concept attacks (Concourse, Thanos).	287 Helm charts from 6 organisations: Bitnami 158, Banzai Cloud 51, Wikimedia 27, Prometheus Community 25, EEA 19, CNCF 10.	+ 634 misconfigurations found in 259 of 287 apps (90%). Only approach that covered all types, versus 11 other tools. Fixes landed in 30+ apps after disclosure. – No severity scoring. Only default-config charts. About 8% UDP false positives had to be removed. Attacks not run on all charts. One app at a time per cluster.	High relevance. Your graph has EXPOSES edges but no pod-to-pod reachability. Add NetworkPolicy nodes and reachability edges, since without a policy the cluster network is flat. Label-selector collisions give impersonation edges. Their threat model (attacker inside a container) matches yours.
7	KubeFence: Security Hardening of the Kubernetes Attack Surface (Cesarano & Natella), arXiv 2504.11126 (Apr 2025), Univ. of Naples Federico II. DSN 2025.	Helm chart → values schema → configuration-space exploration → helm template rendering → YAML "validator". Enforced by a Mitmproxy proxy in front of the API server. Compared against RBAC policies generated with audit2rbac. K8s 1.28.6 testbed.	Runtime, workload-specific filtering of Kubernetes API requests, to cut attack surface and block CVE and misconfiguration exploits.	49 K8s CVEs (2016–2023). 6,580 e2e tests. 5 Artifact Hub operators (PostgreSQL, Nginx, MLflow, RabbitMQ, SonarQube). A catalog of 15 malicious specs (8 CVE-based, 7 misconfigurations).	+ Blocked 15/15 attacks where RBAC blocked 0. About 35% more attack-surface reduction than RBAC. Added only 27–85 ms latency. Open source. – Helm-only. Possible false positives for unusual workloads. Proxy adds latency. Residual risk where legitimate use needs a field. Does not cover supply-chain attacks. Only 5 operators.	Medium. It is prevention, while yours is detection and prioritisation. Its main point is that RBAC cannot see spec fields, which supports scoring Pod specs directly. Its list of 15 risky fields is a checklist. You do not yet cover hostNetwork, externalIPs, subPath, seccomp or seLinuxOptions.

#	Paper detail	Tools & techniques	Applications	Dataset	Advantages / Disadvantages	Inference for current project
8	Take Over the Whole Cluster: Attacking Kubernetes via Excessive Permissions of Third-party Applications (Yang et al.), ACM CCS '23.	Three attack strategies chaining excessive RBAC permissions. A script finds "critical" DaemonSets and components. Attacks are launched manually.	Auditing add-ons and RBAC. Led to 8 CVEs and a Google bounty.	153 CNCF projects, 51 of them (33.3%) vulnerable. Add-ons from GKE (14), EKS (13), AKS (7) and Alibaba ACK (13).	+ Real, vendor-confirmed findings. – Semi-manual. Assumes the attacker already controls a worker node.	High relevance. This is an attack path, not a single finding. It chains patch nodes (force a pod onto your node) with list secrets (steal the admin token). Add these verbs to your RBAC categories: patch/update nodes, delete pods, escalate/bind clusterroles, list/update clusterrolebindings and *. Model "can reschedule pod X" as an edge. Their fixes (resourceNames, RoleBinding over ClusterRoleBinding) make good remediation advice.
9	Exploiting Kubernetes' Image Pull Implementation to Deny Node Availability (Knob, Franzil, Siracusa), arXiv 2401.10582 (Jan 2024, v2 Aug 2025), Fondazione Bruno Kessler / Univ. of Trento.	Attack analysis of the asynchronous CRI-API (create a pod, then force-delete it so the image pull continues). Experiments on a local 3-VM cluster (K8s v1.27, containerd) and a 10-node GKE cluster. Custom "Scheduling Delay" metric and Phoronix benchmarks. Mitigation MAGI: eBPF plus audit logs that detect and kill the pull.	Resource-exhaustion DoS against worker nodes, and manipulation of the node image cache. Threat modelling for edge and multi-tenant clusters.	Two self-built image sets (7 images of 2–14 GB; 40 images of 0.1–0.9 GB), plus real ML images on GKE. No public dataset.	+ Shows a new attack that RBAC, Pod Security Standards and quotas do not stop. Up to 95% average CPU, kernel build time +91%, stress-ng throughput −33%. Open-source mitigation, responsibly disclosed. – Maintainers classed it as a resource-usage bug, not a vulnerability. Needs permission to create and delete pods (or patch a Deployment). MAGI needs audit logging, root and a kubelet restart, is fragile across versions, and cannot block small images.	Manifest and Helm analysis cannot see this class of runtime and control-plane threat. If your project builds attack graphs, add it as an optional DoS node and note it as out of scope for static checks.

#	Paper detail	Tools & techniques	Applications	Dataset	Advantages / Disadvantages	Inference for current project
10	Kubernetes Autoscaling: YoYo Attack Vulnerability and Mitigation (Ben David, Bremler-Barr), CLOSER 2021.	Burst-DDoS ("YoYo") attack on the pod and node autoscalers in Google Kubernetes Engine (GKE). Detection with XGBoost on response time, pod count, node count and CPU load.	Protecting autoscaling clusters from economic and performance DoS.	The authors' own GKE runs with JMeter load. 21 attack samples plus regular load, 70/30 split.	+ Costs the attacker about a third of a flat DDoS. XGBoost reaches about 94–95% accuracy and beats CNN+LSTM. – No production data. Single application, not multi-tenant.	Tangential. It covers availability, which your tool does not. At most it supports a resource-limits or autoscaling risk factor. XGBoost is a possible baseline against your GCN.
11	Graphene: Infrastructure Security Posture Analysis with AI-generated Attack Graphs (Jin et al.), arXiv 2312.13119v2, 2024.	NER (spaCy) extracts preconditions and postconditions from CVE text. Word embeddings (CBOW and Skip-Gram) match them into edges. Edge exploitability, impact and risk scores are normalised.	Attack-graph security scoring for IT and IoT infrastructure.	NER set of 52,532 CVEs with 245,573 labelled entities. 62,544 documents for embeddings. Case study on an IoT testbed.	+ Fully automated. Scoring can be customised per deployment. – The embedding step is evaluated only visually (t-SNE). Not Kubernetes-specific. Attack graphs can grow very large.	High relevance. It is the closest match to your design: attack graph plus risk scoring. Model edges as precondition and postcondition (for example, "holds ServiceAccount token" enables "list secrets"). Normalise edge scores and let users pass custom scoring functions. Your graph uses RBAC and config where theirs uses CVEs, so image CVEs (such as Trivy) could be a later node attribute.
12	GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation (Malul et al.), arXiv 2405.19954, 2024.	Fine-tuned CodeT5p 770M for detection. Mistral for explanation and remediation. Labels come from 3 rule-based tools (Checkov, KubeLinter, Terrascan) through a unified index of 169 misconfigurations.	Detecting and fixing misconfigurations in manifests.	229,989 labelled manifests from The Stack, plus 232 Kubernetes documents (700 MB).	+ Precision 0.990 and recall 0.999, better recall than any single rule-based tool. Expert judged its explanations 100% correct. – 512-token input limit. Labels depend on rule-based tools. Adapting to new misconfigurations takes retraining.	Medium. It is an ML approach to the same problem, and an LLM could generate your remediation text. It supports your argument about the limits of rule-based tools. A GCN trained on rule-based pseudo-labels carries the same ceiling, which your manual-label feature helps address.

#	Paper detail	Tools & techniques	Applications	Dataset	Advantages / Disadvantages	Inference for current project
13	Co-dependence Aware Fuzzing for Dataflow-Based Big Data Analytics (DepFuzz) (Humayun, Kim, Gulzar), ESEC/FSE '23.	Fuzzing with taint tracking at row, column and dataset level. Mutates co-dependent input regions together. Compared against Jazzer and BigFuzz.	Testing Spark and Hadoop data pipelines.	17 Spark programs, 24-hour runs, 45 injected faults.	+ 29% more coverage than Jazzer, 3.4× more faults. – One-time instrumentation overhead of 1.1–14×. Scala/Spark only. Not Kubernetes.	Low relevance. Its fault-injection method could shape a testing section for your tool. Check whether this link belongs in the list.
14	Helm-ET: Reducing Exposure to Lateral Movement in Kubernetes Artifacts (Bufalino et al.), IEEE CLOUD 2025. Focuses on automatically generating Kubernetes NetworkPolicies to reduce lateral movement.	Kubernetes, Helm charts, NetworkPolicy, service-composition patterns, modularity/information hiding, automatic policy generation.	Reducing internal Kubernetes attack surface and restricting unnecessary pod-to-pod communication.	451 Kubernetes applications/Helm charts across datasets: Artifact Hub, OSS applications and DeathStarBench. The Artifact Hub dataset contains 446 charts.	+ Automatically generates policies; significantly reduces allowed connections; fast policy generation; policies can be applied before deployment. Paper reports 92.71% reduction in allowed connections and generation below 100 ms. – Focuses mainly on network-layer lateral movement; NetworkPolicy has limitations and cannot represent every application-level interaction. It does not directly analyze RBAC privilege escalation or complete attack chains.	Highly relevant. Network relationships can become another edge type in your attack graph. Your graph can combine Pod → Service → Network access → RBAC privilege rather than treating configuration findings independently.
15	A Systematic Mapping Study on Risks and Vulnerabilities in Software Containers (Sroor et al.). A broad study categorizing security risks and vulnerabilities throughout the container lifecycle.	Systematic Mapping Study, taxonomy construction, qualitative analysis, risk/vulnerability classification, mitigation analysis, security-tool analysis.	Understanding container security across image, host, application, network, orchestration and runtime phases.	Literature-based dataset. 129 relevant primary studies analyzed; searches covered IEEE Xplore, ACM, Scopus, Web of Science, ScienceDirect and other databases.	+ Very broad coverage; provides taxonomy of risks, causes, impacts, mitigation techniques and security tools. Useful as a foundation for identifying categories of Kubernetes/container risks. – Not a runtime Kubernetes attack-graph system. Being literature-based, it does not provide direct cluster topology or attack-path data. Authors also identify possible database/search-string coverage limitations.	Use this paper to justify your risk categories. Your project can narrow the broad container-security taxonomy into Kubernetes-specific risks such as misconfiguration, exposure, RBAC, privilege escalation and lateral movement.

#	Paper detail	Tools & techniques	Applications	Dataset	Advantages / Disadvantages	Inference for current project
16	Securing the Open RAN Infrastructure: Exploring Vulnerabilities in Kubernetes Deployments (Klement et al.), IEEE NetSoft 2024. Examines security weaknesses in Kubernetes infrastructure supporting the O-RAN RIC.	Static security scanning, MITRE ATT&CK, NSA/CISA guidance, CIS benchmarks, vulnerability assessment and quantitative severity assessment.	Security assessment and hardening of Kubernetes-based Open RAN infrastructure.	Near-RT RIC / O-RAN Software Community Kubernetes deployment. The study reports 792 vulnerabilities and 70 CVEs in the examined platform versions.	+ Real-world Kubernetes deployment; combines multiple security frameworks; demonstrates practical impact of outdated packages and insecure configurations. – Domain-specific to Open RAN; mainly performs security assessment/scanning rather than modeling chained attack paths. Findings can become large lists without graph-based relationship analysis.	Supports your configuration + CVE integration. Your project can take scanner findings such as CVEs/configuration weaknesses and connect them to Kubernetes objects, then determine whether they form an actual attack path.
17	Towards Secure Cloud-Native Computing: Unveiling Kubernetes Misconfigurations with Large Language Models (Ghorab & Saied); arXiv 2609.20834v1 appears to be the same study (unified misconfiguration taxonomy and LLM-based detection). Compares ML/LLMs with existing tools.	BERT, CodeBERT, RoBERTa, LLaMA3; supervised ML/LLM classification with precision/recall evaluation; taxonomy. Compared against rule-based tools: Kube Score, Snyk, Datree, Checkov, KubeLinter, Terrascan.	Automated Kubernetes misconfiguration detection, identification and classification.	Four constructed datasets: 13,723 / 26,984 / 32,956 / 276,520 entries. Dataset 4 uses object-level organization and injected (synthetic mutation) misconfigurations to address imbalance.	+ ML/LLMs can capture patterns that static rules may miss; large datasets; broad coverage; comparison against multiple established tools. – ML/LLM performance decreases for complex RBAC and NetworkPolicy cases; contextual information is important. Rule-based tools have static-rule limits and false positives/negatives.	Very relevant to your planned GNN/ML risk-scoring component. RBAC is difficult for ML models, which supports keeping a rule-based security layer alongside ML. Strongly supports a hybrid architecture: use deterministic rules for known dangerous configurations/RBAC permissions, then graph/ML techniques to assess combined risk and attack-path significance.

#	Paper detail	Tools & techniques	Applications	Dataset	Advantages / Disadvantages	Inference for current project
18	Breaking the Bulkhead: Demystifying Cross-Namespace Reference Vulnerabilities in Kubernetes Operators (Chen et al.), NDSS 2026. Studies a new Kubernetes Operator vulnerability that can break namespace isolation and cause privilege escalation.	Static analysis, Kubernetes Operator analysis, namespace-scope analysis, RBAC analysis, attack strategies for cross-namespace references.	Detecting Operator vulnerabilities, namespace isolation violations and privilege-escalation opportunities.	Large-scale measurement of Kubernetes Operators in the wild. Over 14% reported as potentially vulnerable; findings resulted in confirmed vulnerabilities and CVEs.	+ Directly addresses privilege escalation and RBAC/namespace relationships; demonstrates that permissions alone may not reveal the complete security problem. – Focuses specifically on Operators and cross-namespace references; does not provide a general-purpose Kubernetes attack-risk graph covering all resource types.	Extremely relevant to your attack graph. Add Namespace, Operator, and cross-namespace permission/reference relationships. This can create realistic paths such as Compromised Pod → ServiceAccount → Operator → Cross-Namespace Resource → Privilege Escalation.
19	Analyzing and Mitigating (with LLMs) the Security Misconfigurations of Helm Charts from Artifact Hub (Minna, Massacci & Tuma), Empirical Software Engineering 2025. Evaluates Helm-chart misconfiguration detection and LLM-based remediation.	Checkov, KICS, Kubeaudit, Terrascan and other static analyzers; LLMs including GPT, Gemini, Claude and Llama; YAML analysis and automated refactoring.	Detecting and automatically fixing security misconfigurations in Helm charts.	Artifact Hub Helm charts, with thousands analyzed; manual evaluation used subsets such as 90 charts for Claude/GPT and 30 for Gemini/Llama.	+ Compares multiple tools; identifies overlapping and unique policies; demonstrates that LLMs can generate fixes. Claude performed particularly well in the reported experiments. – LLM fixes can unintentionally remove valid configuration or introduce other problems. Performance varies significantly between models and misconfiguration types.	Useful for your future remediation feature. Your current project should first identify and prioritize attack paths; later, an LLM could explain the risk and suggest remediation for the specific Kubernetes configuration causing the path.

#	Paper detail	Tools & techniques	Applications	Dataset	Advantages / Disadvantages	Inference for current project
20	Kubernetes Misconfigurations in the Wild: Taxonomy, Evolution, and Automated Repair with Large Language Models (Ghorab, Abdel Latif & Saied), 2026. Studies real-world Kubernetes security misconfigurations and automated correction.	Stack Overflow mining, taxonomy construction, BERTopic/BERT embeddings, LLM-based correction, Kubecurity schema-guided validation.	Taxonomy creation, severity analysis and automated remediation of Kubernetes misconfigurations.	2,662 developer-reported issues from Stack Overflow; additional Kubernetes configuration data from GitHub/other sources used for evaluation.	+ Uses real-world developer problems; contextual LLM reasoning improves correction; hybrid LLM + deterministic schema validation achieved 98.50% correction accuracy according to the paper. – Primarily focuses on misconfiguration detection and repair, not attack-path discovery. LLM-based correction requires sufficient context and validation to avoid introducing new problems.	Supports a future explainable remediation layer. Instead of simply saying "high risk," your dashboard could explain why the path exists and which configuration/RBAC rule should be changed.

2.2 RESEARCH GAPS

- Individual configuration findings do not always communicate the relationship between exposure, identity and privilege.
- Security analysts need a concise representation of attack chains instead of manually correlating multiple scanner outputs.
- Risk scoring can become difficult to explain when it depends entirely on an opaque model; a transparent baseline is useful for validation and viva-level explanation.
- Small project datasets make training a fully supervised security GNN difficult because reliable labels are expensive and incident datasets may not match the target cluster.
- An end-to-end educational system needs a reproducible local environment, controlled vulnerable scenarios and a dashboard that demonstrates each stage.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW

The proposed K8s Attack Graph system converts Kubernetes configuration into an analyzable security graph and combines deterministic rules with graph-based learning. The pipeline starts with cluster discovery and ends with an interactive dashboard. The implementation intentionally follows a fallback-friendly architecture: if the GNN is unavailable or insufficiently trained, the rule-based risk score remains usable.

2.3.2 OBJECTIVES

- Automate extraction of security-relevant Kubernetes configuration.
- Expose relationships between workload identity, RBAC, services and deployments.
- Identify paths to dangerous permissions and categorize their severity.
- Provide explainable risk breakdowns for each Pod.
- Use graph structure as an additional signal for risk re-ranking.
- Present results through a practical web dashboard suitable for security review and demonstration.

2.3.3 ADVANTAGES

- Relationship-centric analysis instead of isolated checklist output.
- Attack paths can be displayed as explicit step-by-step chains.
- Rule-based scores are transparent and decomposable into contributing factors.
- Neo4j and NetworkX make graph inspection and traversal straightforward.

- Redis avoids repeating cluster discovery between pipeline stages.
- The GNN is optional and does not replace the explainable baseline.
- The demo cluster includes both vulnerable and safe workloads, enabling a simple control comparison.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The architecture is organized as a sequence of services and analysis modules. Kubernetes/Minikube is the source environment. The Kubernetes Python client extracts relevant objects into project-specific dataclasses. Redis stores the most recent discovery result. The graph builder writes resource nodes and relationships to Neo4j. The attack-path engine loads the graph into NetworkX. The risk engine calculates per-Pod scores, and the GNN module optionally re-ranks them. A React frontend communicates with the FastAPI backend and visualizes the resulting data.

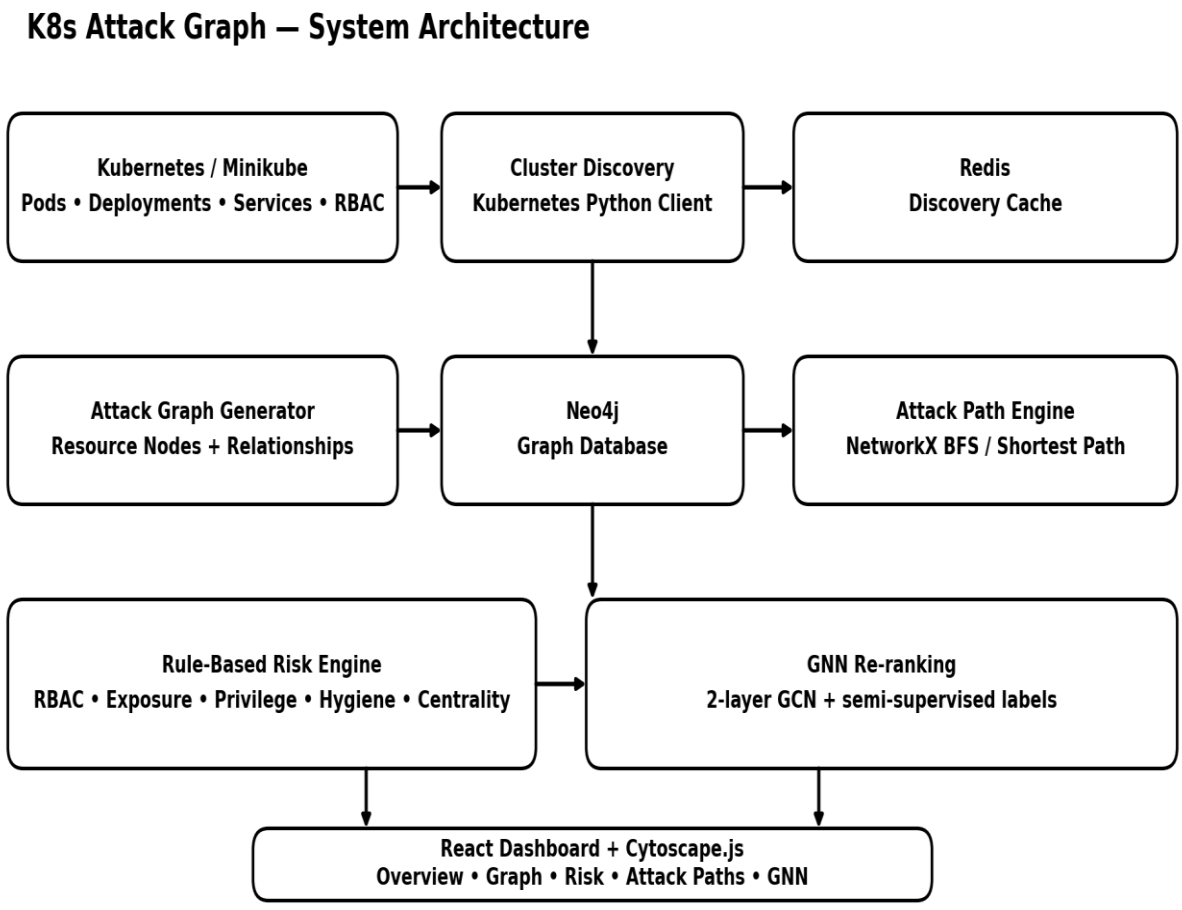


Figure 3.1 System Architecture

3.2 MODULE DESIGN

TABLE 3.1 : PROJECT MODULES, TECHNOLOGIES, AND RESPONSIBILITIES

Module	Technology	Main Responsibility
Cluster Discovery	Kubernetes Python client	Collect Pods, Deployments, Services, ServiceAccounts, Roles, ClusterRoles and bindings.
Data Cache	Redis	Store the latest serialized DiscoveryResult between pipeline stages.
Attack Graph Generator	Python + Neo4j	Create nodes and RUNS_AS, BOUND_TO, EXPOSES and MANAGES relationships.
Attack Path Engine	NetworkX	Build a directed in-memory graph and run shortest-path and notable-path analysis.
Risk Engine	Python + NetworkX	Calculate transparent 0–100 Pod scores from five weighted factors.
GNN Re-ranking	PyTorch	Run a small two-layer GCN using graph features and combined labels.
Backend API	FastAPI	Expose discovery, graph, attack-path, risk and GNN endpoints.
Dashboard	React + Cytoscape.js	Provide overview, graph, risk, attack-path and GNN comparison pages.

3.2.1 CLUSTER DISCOVERY MODULE

The discovery client first attempts in-cluster configuration and falls back to the local kubeconfig, which is the normal Minikube development path. It extracts only fields needed for security analysis. For Pods, these include namespace, node, ServiceAccount, hostNetwork, privileged containers, hostPID, hostIPC, ServiceAccount token mounting, non-root enforcement and image tagging. RBAC objects are converted into simple rule dictionaries so later modules do not depend directly on raw Kubernetes client objects.

3.2.2 ATTACK GRAPH GENERATOR

The graph builder recreates the Neo4j graph from the current discovery result to avoid stale relationships. Pod and Deployment nodes are associated with ServiceAccounts through RUNS_AS. ServiceAccounts are connected to Roles or ClusterRoles through BOUND_TO. Services are connected to matching Pods through EXPOSES, and Deployments are connected to Pods through best-effort label matching using MANAGES.

3.2.3 ATTACK PATH ENGINE

The Attack Path Engine reads Neo4j nodes and edges into a NetworkX directed graph. A shortest-path function can find a path between any two known node identifiers. A focused critical-path search examines exposed services leading to cluster-admin-equivalent permissions, while the broader notable-path search checks paths from Pods to dangerous permission patterns including wildcard permissions, privilege-escalation verbs, pod exec and secrets access.

3.2.4 RISK ENGINE

The rule-based engine produces a 0–100 score for each Pod. The design uses five factors: RBAC escalation potential, network exposure, container/host privilege, identity/image hygiene and graph centrality. The weights are 35, 25, 20, 10 and 10 respectively. The resulting score is categorized as Critical (70–100), High (40–69), Medium (15–39) or Low (0–14). The implementation sorts Pods by score and returns a detailed breakdown so the analyst can see why a Pod received its score.

3.2.5 GNN RE-RANKING MODULE

The GNN is a two-layer Graph Convolutional Network implemented directly with PyTorch. Node features include a one-hot resource type, security flags and normalized degree. The graph is symmetrized for message passing and self-loops are added before normalized adjacency is calculated. Manual ground-truth labels are given higher training weight than rule-based pseudo-labels. The model is retrained on demand because the demonstration graph and labels can change after discovery.

3.2.6 FRONTEND AND API MODULE

The FastAPI backend exposes endpoints for health, discovery, graph data, attack paths, risk scores and GNN re-ranking. The React/Vite frontend provides pages for

Overview, Graph View, Risk Table, Attack Paths and GNN Compare. Cytoscape.js is used for interactive graph visualization.

3.3 DATA FLOW

The following functional flow represents the implemented pipeline and the interaction between the analyst, Kubernetes cluster, backend services, graph store and dashboard.

TABLE 3.2 : SYSTEM COMPONENTS, INTERACTIONS, AND OUTPUTS

Actor / Component	Interaction	Output
Analyst	Starts discovery from dashboard/API	Discovery request
Kubernetes API	Returns resource inventory	Pods, Services, RBAC, Deployments
Redis	Stores serialized discovery result	Cached DiscoveryResult
Graph Builder	Transforms inventory into graph	Neo4j nodes and relationships
NetworkX Engine	Traverses graph	Attack paths
Risk Engine	Scores Pods	0–100 scores + breakdown
GNN Module	Refines risk ordering	Rule vs GNN comparison
Dashboard	Visualizes outputs	Security analysis views

The flowchart shows the tool's pipeline: it discovers the cluster with the Kubernetes Python client, caches the result in Redis, builds an attack graph in Neo4j, traverses attack paths with NetworkX, computes rule-based risk scores (0-100), optionally trains and re-ranks with a GNN, and finally displays the results on a React/Cytoscape.js dashboard.

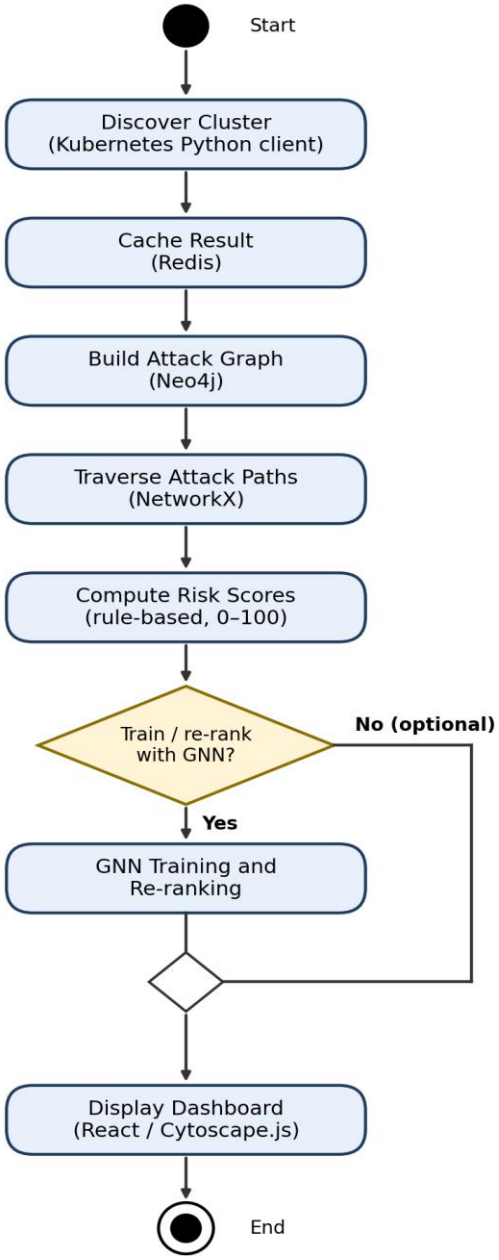


Figure 3.2 Activity Flow of the Analysis Pipeline

3.4 THREAT MODEL AND ATTACK SURFACE

The threat model assumes that an attacker may gain an initial foothold in or reach an exposed workload and then attempt lateral movement or privilege escalation. The graph focuses on the configuration relationships that can turn a local foothold into broader permissions. The attack surface includes externally exposed Services, ServiceAccounts, RoleBindings/ClusterRoleBindings, wildcard permissions, privilege-escalation verbs, pod exec permissions, secrets access, privileged containers and host namespace settings.

TABLE 3.3: THREATS, GRAPH SIGNALS, AND POTENTIAL SECURITY EFFECTS

Threat / Misconfiguration	Relevant Graph Signal	Potential Security Effect
Public service + powerful ServiceAccount	EXPOSES + RUNS_AS + BOUND_TO	Remote-to-RBAC escalation chain
Cluster-admin binding	BOUND_TO → ClusterRole(cluster-admin)	Broad cluster control
Wildcard permissions	Role/ClusterRole property	Unrestricted resource/action scope
Privilege-escalation verbs	RBAC rule properties	Potential escalation through RBAC
Pod exec capability	RBAC rule property	Command execution in another workload
Secrets get/list	RBAC rule property	Credential/data exposure
Privileged container	Pod property	Reduced container isolation
hostNetwork / hostPID / hostIPC	Pod properties	Increased host/process/IPC exposure
Auto-mounted ServiceAccount token	Pod property	Credential exposure from workload
Unpinned/latest image	Pod property	Image supply-chain hygiene concern

3.5 SECURITY CONTROLS

- Least-privilege RBAC should be applied to the discovery identity.
- Network exposure should be minimized; NodePort and LoadBalancer services should be intentional.
- ServiceAccount token mounting should be disabled where not required.
- Containers should explicitly enforce non-root execution and avoid privileged mode.

- Host namespaces should only be enabled when required.
- Container images should be pinned to controlled versions rather than relying on latest/untagged images.
- Administrative bindings should be reviewed for cluster-admin and wildcard permissions.
- Demo credentials and development CORS settings must not be reused unchanged in production.

CHAPTER 4

SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The system collects security-relevant Kubernetes objects through the Kubernetes Python client. The discovery result contains six primary categories: Pods, Deployments, ServiceAccounts, Roles, RoleBindings and Services. The data model deliberately converts Kubernetes API objects into simple Python dataclasses so the graph layer has a stable input structure.

TABLE 4.1 : KUBERNETES RESOURCES, COLLECTED INFORMATION, AND SECURITY RELEVANCE

Resource	Collected Information	Security Relevance
Pod	name, namespace, node, ServiceAccount, hostNetwork, privileged containers, hostPID, hostIPC, image information, token mounting, non-root enforcement	Workload exposure, identity and isolation
Deployment	name, namespace, ServiceAccount, replicas, labels	Workload ownership and relationships
ServiceAccount	name, namespace	Workload identity
Role / ClusterRole	rules, verbs, resources, API groups	Permission and escalation analysis
RoleBinding	subjects and referenced role	Identity-to-permission relationship
Service	type, ports, selector	External exposure and workload reachability

4.2 DATA PREPROCESSING

Preprocessing occurs in two major forms. First, raw Kubernetes objects are normalized into dataclasses and serialized to JSON for Redis caching. Second, the graph builder derives boolean security properties from RBAC rules and Pod settings. For example, a Role is checked for wildcard resources or verbs, privilege-escalation verbs, pod exec access and secrets access. A Pod is checked for privileged containers, host namespaces, automatic ServiceAccount token mounting, non-root enforcement and image tags.

4.3 METHODOLOGY

4.3.1 Graph Construction

Each discovered resource receives a stable identifier such as pod/<namespace>/<name> or clusterrole/-/<name>. The graph builder creates nodes using these identifiers and adds relationships according to the discovered configuration. Rebuilding from scratch ensures that deleted or changed relationships do not remain as stale edges.

4.3.2 Attack Path Search

For a requested source and target, NetworkX shortest_path returns the minimum-hop route if one exists. For notable paths, the engine evaluates paths from each Pod to dangerous Role/ClusterRole nodes. Results are annotated with dangerous categories and whether the entry point is externally exposed. This lets the dashboard distinguish an internet-facing escalation chain from an internal-only path.

4.3.3 Rule-Based Risk Scoring

TABLE 4.2 : RULE-BASED KUBERNETES RISK SCORING FRAMEWORK

Factor	Maximum Score	Implementation Logic
RBAC escalation potential	35	Highest matched danger level: cluster-admin/wildcard 35; privilege-escalation verbs 28; pod exec 22; secrets access 16; limited role binding floor 6.
Network exposure	25	25 if an externally exposed Service points to the Pod; otherwise 0.
Container/host privilege	20	Privileged container 10; hostNetwork 4; hostPID 4; hostIPC 2.
Identity/image hygiene	10	ServiceAccount token auto-mount 3; missing non-root enforcement 4; latest/untagged image 3.
Graph centrality	10	Normalized betweenness centrality scaled to 0–10 as a blast-radius proxy.

The total is capped at 100. The categories are derived directly from the implementation: Critical ≥70, High ≥40, Medium ≥15 and Low below 15. The centrality component uses betweenness centrality to approximate how consequential a Pod may be in the graph.

4.3.4 GCN Re-ranking

The GNN feature vector contains six one-hot resource-type features, twelve security flags and one normalized degree feature, giving 19 input dimensions. The model uses two hidden graph-convolution layers followed by a linear output layer and sigmoid scaling to 0–100. For training, manually assigned ground-truth labels receive a higher loss weight than rule-based pseudo-labels. The implementation also supports a pretrained checkpoint generated from RBAC Atlas examples; if the checkpoint is unavailable or incompatible, training falls back to random initialization.

4.4 RESULTS AND DISCUSSION

The project repository documents an end-to-end demonstration against a live Minikube cluster. The supplied screenshots show the complete dashboard pipeline, including the overview, a graph containing 148 nodes and 66 edges on the demo cluster, the risk table, attack-path results and GNN re-ranking.

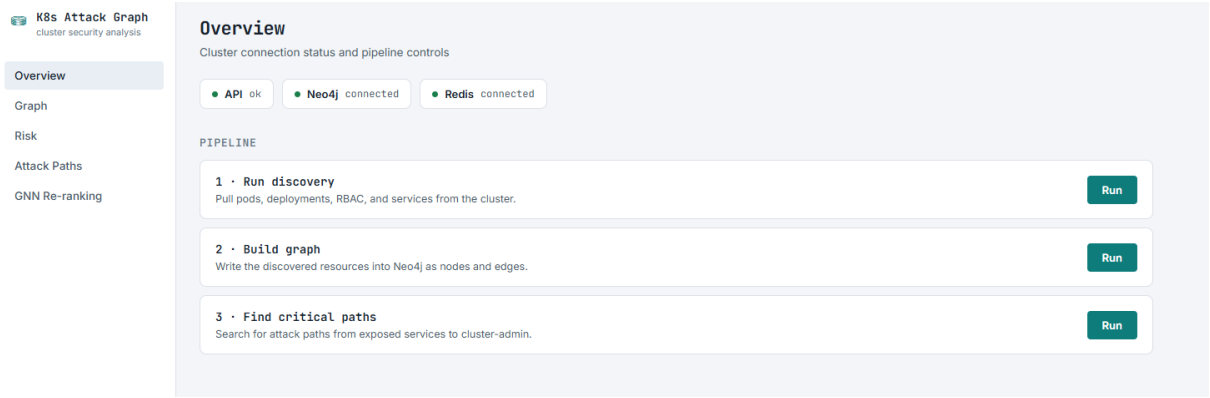


Figure 4.1 Dashboard Overview

The Overview page acts as the operational entry point. It provides pipeline controls for discovery and graph construction and indicates the current state of the analysis. This reduces the number of manual steps required during a demonstration.



Figure 4.2 Generated Kubernetes Attack Graph

The Graph page visualizes the cluster inventory as interconnected nodes. The documented demo view contains 148 nodes and 66 edges. The graph can be filtered by namespace and node information can be inspected through the dashboard.

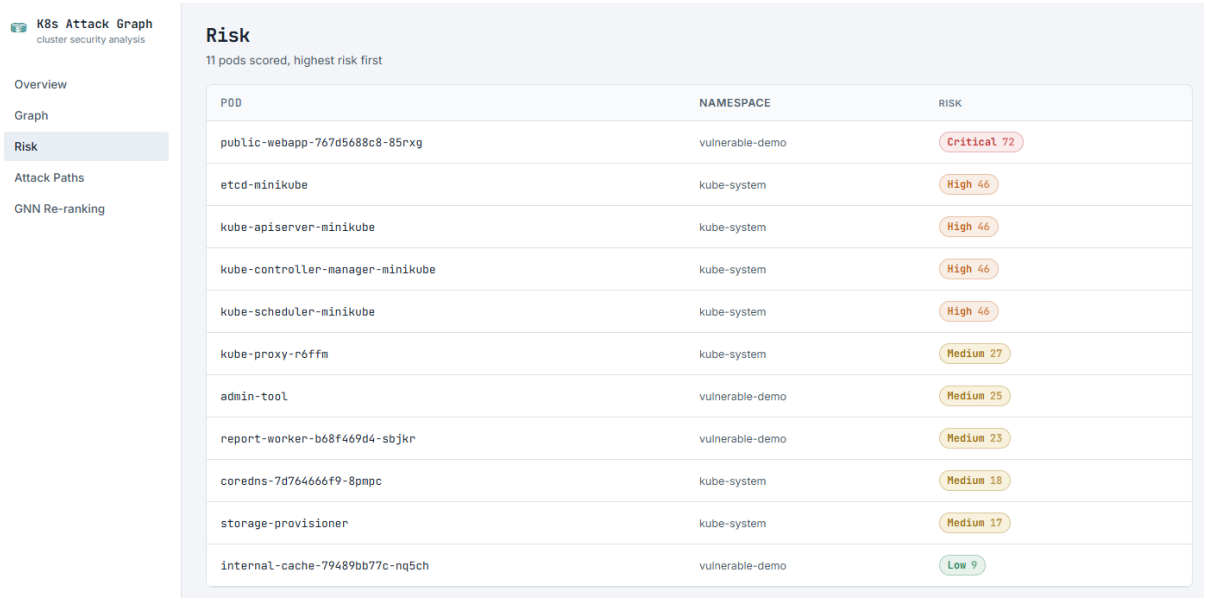


Figure 4.3 Pod Risk Analysis

The Risk page presents a sortable list of Pods with 0–100 scores and severity categories. The expandable breakdown corresponds directly to the five scoring factors, making the result explainable rather than a single unexplained number.

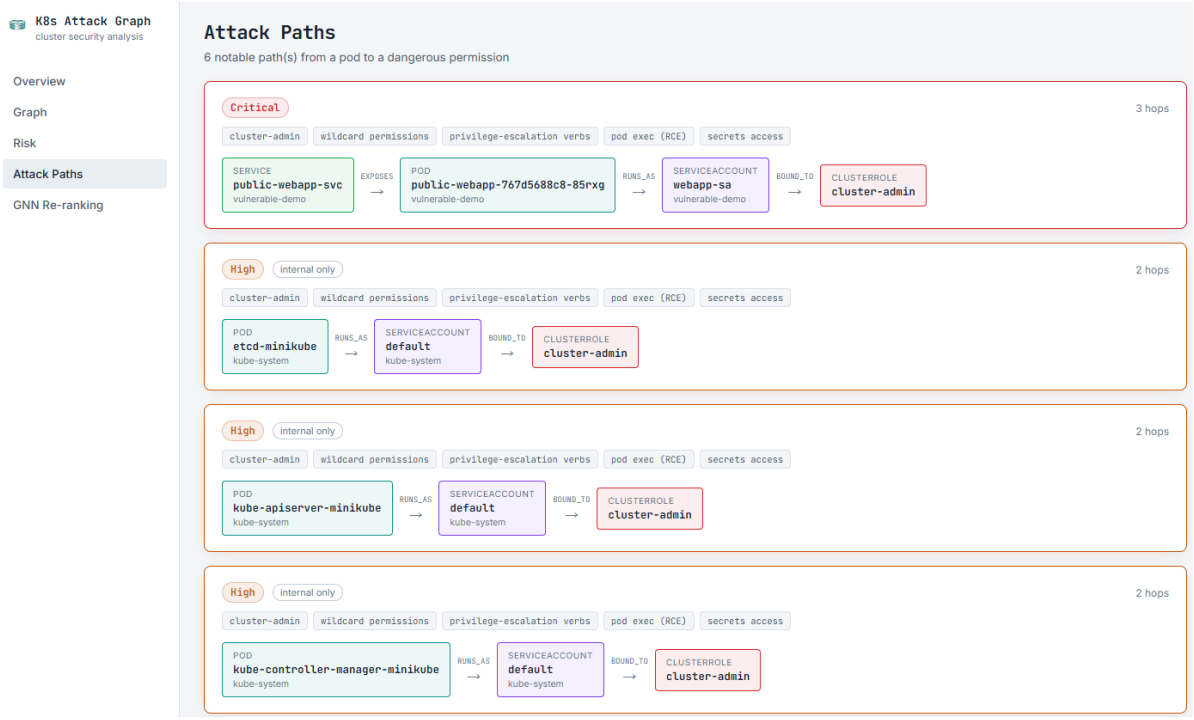


Figure 4.4 Attack Path View

The Attack Paths page displays notable privilege-risk chains as step-by-step sequences. The implementation identifies dangerous patterns such as cluster-admin, wildcard permissions, privilege-escalation verbs, pod exec and secrets access. It also records whether the entry point is externally exposed.

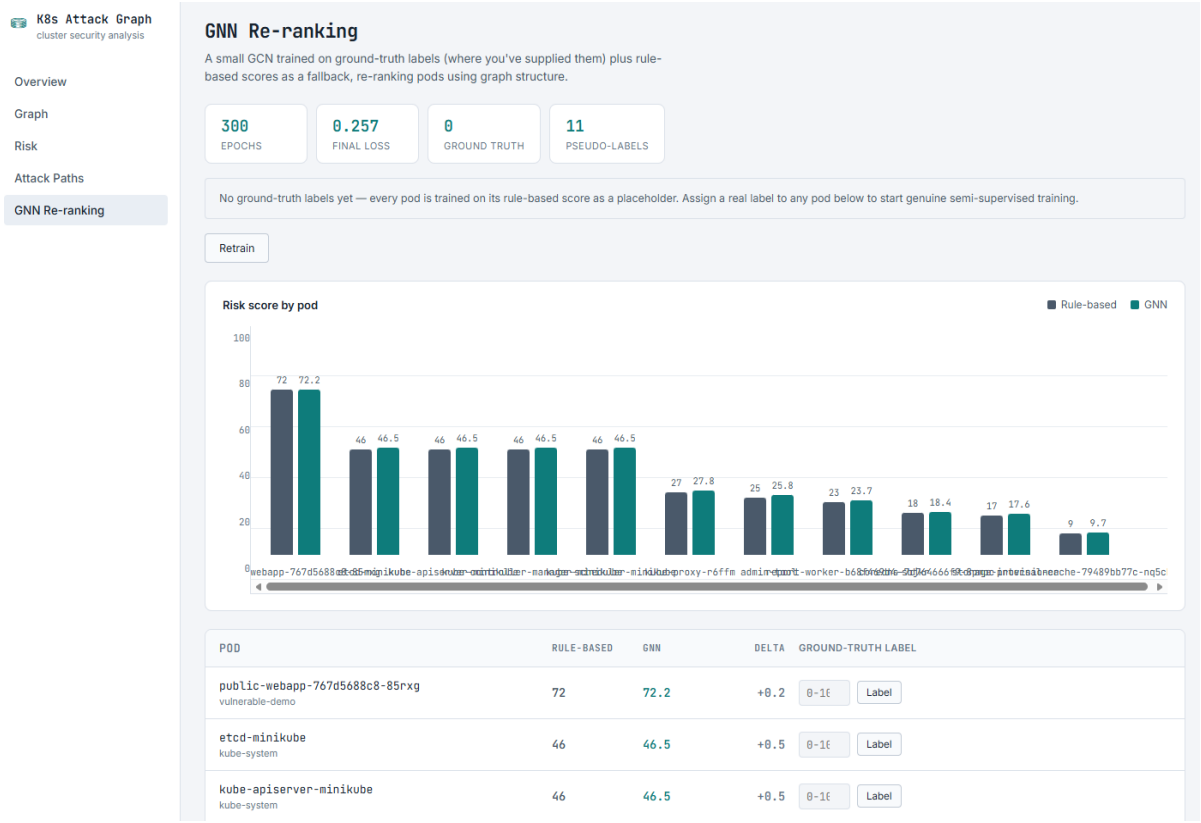


Figure 4.5 GNN Re-ranking View

The GNN page compares rule-based and GNN-derived scores and reports training diagnostics. The UI supports adding ground-truth labels for Pods. This is important because the project deliberately avoids claiming that a small pseudo-labeled dataset alone establishes general predictive performance.

4.4.1 Demo Cluster Scenarios

TABLE 4.3 : DEMO CLUSTER SCENARIOS

Scenario	Configuration	Expected Analysis
Public web app	ServiceAccount bound to cluster-admin and externally exposed service	High-priority privilege-risk path from exposed service to broad RBAC permissions.
Privileged container	Privileged container with hostNetwork and hostPID	High configuration risk due to reduced isolation and host-level access.
Secrets over-exposure	Worker Pod with namespace-wide secrets get/list	RBAC risk through sensitive data access.
Safe control case	Properly scoped, non-exposed Pod	Low-risk comparison case demonstrating that the tool does not simply flag every workload.

These workloads were intentionally created only for a local/throwaway Minikube environment. The project documentation explicitly warns against deploying them to production or any shared environment.

4.4.2 Observed End-to-End Outputs

TABLE 4.4 : OBSERVED END-TO-END OUTPUTS

Component	Observed / Documented Output
Cluster discovery	Resource inventory for Pods, Deployments, ServiceAccounts, Roles, RoleBindings and Services.
Neo4j graph	Demo screenshot documents 148 nodes and 66 edges.
Attack-path analysis	Critical and broader notable privilege-risk paths are presented as chains.
Risk analysis	Each Pod receives a transparent 0–100 score with factor-level breakdown.
Dashboard	Overview, Graph, Risk, Attack Paths and GNN Compare pages are implemented.
GNN	Rule-based and GNN scores are displayed side by side; manual ground-truth labeling is supported.

CHAPTER 5

CONCLUSION

The K8s Attack Graph project successfully implements an end-to-end graph-based security analysis workflow for Kubernetes. Instead of stopping at individual configuration findings, the system connects workloads, identities, permissions and services into an attack graph and uses graph traversal to expose meaningful privilege-risk paths. The rule-based risk engine complements this graph analysis by producing transparent per-Pod scores based on RBAC escalation potential, exposure, container/host privilege, hygiene and centrality.

The project also demonstrates how machine learning can be introduced without making the entire security decision process dependent on an opaque model. The GCN operates as a re-ranking layer above the rule-based baseline and can incorporate analyst-provided labels. This architecture provides a practical fallback and makes the system easier to explain, test and defend.

The dashboard completes the implementation by turning the underlying services into a demonstrable security-analysis workflow. The local Minikube scenarios provide a controlled environment containing both intentionally vulnerable and safe workloads, allowing the discovery, graph, attack-path, risk and GNN components to be exercised together.

Future development can extend the project by integrating image/CVE scanning, network-flow information, more comprehensive Kubernetes resource coverage, richer temporal analysis, larger labeled datasets, stronger GNN evaluation protocols and automated remediation recommendations. Such extensions should be added while preserving the current explainable rule-based baseline and the separation between observation, risk analysis and remediation.

REFERENCES

1. Abdel Latif, Ghorab, & Saied (2026). Kubernetes Misconfigurations in the Wild: Taxonomy, Evolution, and Automated Repair with Large Language Models.

2. Ben David & Bremler-Barr (2021). Kubernetes Autoscaling: YoYo Attack Vulnerability and Mitigation. In *Proc. CLOSER*.

3. Blaise & Rebecchi (2022). Stay at the Helm: Secure Kubernetes Deployments via Graph Generation and Attack Reconstruction. In *Proc. IEEE CLOUD*.

4. Bufalino et al. (2025). Helm-ET: Reducing Exposure to Lateral Movement in Kubernetes Artifacts. In *Proc. IEEE CLOUD*.

5. Bufalino, Martin-Navarro, Di Francesco, & Aura (2025). Inside Job: Defending Kubernetes Clusters Against Network Misconfigurations. arXiv:2506.21134.

6. Cesarano & Natella (2025). KubeFence: Security Hardening of the Kubernetes Attack Surface. arXiv:2504.11126 (DSN 2025).

7. Chen et al. (2026). Breaking the Bulkhead: Demystifying Cross-Namespace Reference Vulnerabilities in Kubernetes Operators. In *Proc. NDSS*.

8. Dell'Immagine, Soldani, & Brogi (2023). KubeHound: Detecting Microservices' Security Smells in Kubernetes Deployments. *Future Internet*, 15(7), 228.

9. Ghorab & Saied (2026). Towards Secure Cloud-Native Computing: Unveiling Kubernetes Misconfigurations with Large Language Models. arXiv:2609.20834.

10. Humayun, Kim, & Gulzar (2023). Co-dependence Aware Fuzzing for Dataflow-Based Big Data Analytics (DepFuzz). In *Proc. ACM ESEC/FSE*.

11. Jin, Katsis, Sang, Sun, Bertino, Kompella, & Kundu (2024). Graphene: Infrastructure Security Posture Analysis with AI-generated Attack Graphs. arXiv:2312.13119.

12. Klement et al. (2024). Securing the Open RAN Infrastructure: Exploring Vulnerabilities in Kubernetes Deployments. In *Proc. IEEE NetSoft*.

13. Knob, Franzil, & Siracusa (2024). Exploiting Kubernetes' Image Pull Implementation to Deny Node Availability. arXiv:2401.10582.

14. Konsta, Spiga, Lluch Lafuente, & Dragoni (2023). A Survey of Automatic Generation of Attack Trees and Attack Graphs. arXiv:2302.14479.

15. Malul et al. (2024). GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation. arXiv:2405.19954.

16. Minna, Massacci, & Tuma (2025). Analyzing and Mitigating (with LLMs) the Security Misconfigurations of Helm Charts from Artifact Hub. *Empirical Software Engineering*.

17. Rahman, Shamim, Bose, & Pandita (2023). Security Misconfigurations in Open Source Kubernetes Manifests. *ACM Transactions on Software Engineering and Methodology*, 32(4).

18. Shamim, Bhuiyan, & Rahman (2020). XI Commandments of Kubernetes Security: A Systematization of Knowledge. In *Proc. IEEE SecDev*, pp. 58–64. arXiv:2006.15275.

19. Sroor et al. (n.d.). A Systematic Mapping Study on Risks and Vulnerabilities in Software Containers.

20. Yang et al. (2023). Take Over the Whole Cluster: Attacking Kubernetes via Excessive Permissions of Third-party Applications. In *Proc. ACM CCS*.

APPENDIX

A PROJECT DIRECTORY STRUCTURE

The submitted project is organized into frontend, backend, infrastructure/demo and documentation components. The backend contains discovery, graph, attack-path, risk and GNN packages. The frontend contains reusable components and pages for the dashboard.

```
project/
├── backend/
│   ├── app/
│   │   ├── discovery/
│   │   ├── graph/
│   │   ├── attack_paths/
│   │   ├── risk_engine/
│   │   ├── gnn/
│   │   ├── api/
│   │   └── core/
│   └── requirements.txt
├── frontend/
│   ├── src/components/
│   ├── src/pages/
│   ├── src/api/
│   └── package.json
├── demo-cluster/
├── docs/screenshots/
├── docker-compose.yml
└── README.md
```

B ATTACK PATH – SAMPLE CODE

```
import logging

import networkx as nx

from app.core.db import get_neo4j_driver

logger = logging.getLogger(__name__)

# Each dangerous-role check pairs a human-readable label with the boolean
# property (set on Role/ClusterRole nodes by graph/builder.py) that flags
# it. A single role can match more than one — e.g. cluster-admin also
# happens to have secrets access, but we report cluster-admin as the
# category since it subsumes the rest.
DANGEROUS_ROLE_CHECKS = [
    ("cluster-admin", lambda d: d.get("name") == "cluster-admin"),
    ("wildcard permissions", lambda d: d.get("has_wildcard")),
    ("privilege-escalation verbs", lambda d: d.get("has_privesc_verbs")),
    ("pod exec (RCE)", lambda d: d.get("has_pod_exec")),
    ("secrets access", lambda d: d.get("has_secrets_access")),
]

# Categories severe enough that even an internal-only (non-exposed) entry
# point is still worth calling "High" rather than "Medium".
SEVERE_CATEGORIES = {"cluster-admin", "wildcard permissions", "privilege-escalation
    verbs"}

def build_networkx_graph() -> nx.DiGraph:
    """Pulls all nodes/edges out of Neo4j into an in-memory directed graph."""
    driver = get_neo4j_driver()
    graph = nx.DiGraph()

    with driver.session() as session:
```

```
        nodes = session.run("MATCH (n) RETURN n, labels(n) AS labels")
        for record in nodes:
            props = dict(record["n"])
            node_id = props.get("id")
            if not node_id:
                continue
            node_type = record["labels"][0] if record["labels"] else "Unknown"
            graph.add_node(node_id, type=node_type, **props)

        edges = session.run(
            "MATCH (a)-[r]->(b) RETURN a.id AS source, b.id AS target, type(r) AS rel_type"
        )
        for record in edges:
            if record["source"] and record["target"]:
                graph.add_edge(record["source"], record["target"], type=record["rel_type"])

    return graph

def _describe_node(graph: nx.DiGraph, node_id: str) -> dict:
    """Turns a bare node id into a small, human-readable summary for API responses."""
    data = graph.nodes.get(node_id, {})
    return {
        "id": node_id,
        "type": data.get("type", "Unknown"),
        "name": data.get("name", node_id),
        "namespace": data.get("namespace", ""),
    }

def _describe_path(graph: nx.DiGraph, path: list[str]) -> dict:
    """Converts a raw list of node ids into a readable step-by-step chain,
    including the relationship type connecting each pair of steps."""
    steps = [_describe_node(graph, n) for n in path]
    hops = []
    for a, b in zip(path, path[1:]):
```

```
        edge_data = graph.get_edge_data(a, b) or {}
        hops.append(edge_data.get("type", "?"))
    return {"steps": steps, "hops": hops, "length": len(path) - 1}

def find_shortest_path(graph: nx.DiGraph, source_id: str, target_id: str) -> dict | None:
    """Finds the shortest path between two specific nodes, if one exists."""
    try:
        path = nx.shortest_path(graph, source_id, target_id)
    except nx.NodeNotFound:
        return None
    except nx.NetworkXNoPath:
        return None
    return _describe_path(graph, path)

def _dangerous_categories(role_data: dict) -> list[str]:
    return [label for label, check in DANGEROUS_ROLE_CHECKS if check(role_data)]

def _pod_is_exposed(graph: nx.DiGraph, pod_id: str) -> bool:
    """A pod is exposed if any externally-exposed Service points to it via EXPOSES."""
    return _exposing_service(graph, pod_id) is not None

def _exposing_service(graph: nx.DiGraph, pod_id: str) -> str | None:
    """Returns the id of the first externally-exposed Service pointing at this pod, if any."""
    for pred in graph.predecessors(pod_id):
        data = graph.nodes[pred]
        if data.get("type") == "Service" and data.get("externally_exposed"):
            return pred
    return None

def find_notable_paths(graph: nx.DiGraph, max_results: int = 30) -> list[dict]:
    """
    Finds every path from a Pod to a dangerous Role/ClusterRole binding,
    regardless of whether that pod happens to be internet-exposed. Each
    result is tagged with:
```

```
  - categories: which dangerous pattern(s) the target role matched
  - exposed_entry: whether the pod is reachable from outside the cluster
  - severity: Critical / High / Medium, derived from the above

This deliberately goes beyond the original "exposed service to
cluster-admin" search — internal-only privilege escalation paths are
still real findings (lateral movement, insider threat, a compromised
neighboring pod), just lower urgency than an internet-facing one.
"""
pod_nodes = [n for n, d in graph.nodes(data=True) if d.get("type") == "Pod"]
    role_nodes = [n for n, d in graph.nodes(data=True) if d.get("type") in ("Role",
"ClusterRole")]

results = []
for pod_id in pod_nodes:
    for role_id in role_nodes:
        categories = _dangerous_categories(graph.nodes[role_id])
        if not categories:
            continue

        path_info = find_shortest_path(graph, pod_id, role_id)
        if not path_info:
            continue

        exposed_service = _exposing_service(graph, pod_id)
        exposed = exposed_service is not None
        if exposed_service:
            # Prepend the exposing Service so the chain reads
            # "attacker's actual entry point -> ... -> target",
            # matching how a security analyst would narrate it.
            service_step = _describe_node(graph, exposed_service)
            path_info["steps"].insert(0, service_step)
            path_info["hops"].insert(0, "EXPOSES")
            path_info["length"] += 1
```

```
            severe = any(c in SEVERE_CATEGORIES for c in categories)

            if exposed and severe:
                severity = "Critical"
            elif exposed or severe:
                severity = "High"
            else:
                severity = "Medium"

            path_info["categories"] = categories
            path_info["exposed_entry"] = exposed
            path_info["severity"] = severity
            results.append(path_info)

    severity_rank = {"Critical": 0, "High": 1, "Medium": 2, "Low": 3}
    results.sort(key=lambda p: (severity_rank.get(p["severity"], 9), p["length"]))
    return results[:max_results]

def find_critical_paths(graph: nx.DiGraph) -> list[dict]:
    """
    Backward-compatible narrow search: paths from an internet-exposed
    Service to a cluster-admin-or-wildcard role only. Kept as a focused
    subset of find_notable_paths() for callers that specifically want
    just the headline external-escalation finding.
    """
    return [
        p
        for p in find_notable_paths(graph)
         if p["exposed_entry"] and any(c in ("cluster-admin", "wildcard permissions") for c in
    p["categories"])
    ]
```

C DEMONSTRATION PROCEDURE

- Start Minikube and verify that kubectl can access the cluster.
- Start Neo4j and Redis with Docker Compose.
- Run the backend locally when scanning the Minikube API server, following the project README.
- Deploy the intentionally vulnerable demo manifests only to the disposable local cluster.
- Open the dashboard and run Cluster Discovery.
- Build the graph and inspect the graph visualization.
- Open Attack Paths to review exposed-to-dangerous-permission chains.
- Open Risk to inspect factor-level Pod scores.
- Open GNN Re-ranking to compare rule-based and learned scores and, if required, add ground-truth labels.
- Remove the vulnerable namespace and cluster-scoped demo binding after the demonstration.